Terminal Buddy: Technical Build Specification (V3.0 - Expanded & Detailed)

Architect: Fred | **Core:** Rust 1.80+ | **Focus:** Kernel-level Correlation & Deep State Introspection

This document outlines the low-level architectural requirements for the Terminal Buddy daemon (the "Sidecar"). The primary objective of this architecture is to bridge the gap between high-speed user interface rendering and deep, kernel-level system monitoring, enabling the "Teachable Moments" that define the product's cognitive assistance capabilities.

1. Correlation Engine: The Context Synthesizer

The Correlation Engine is the brain of the Sidecar. It does not merely log text; it links a specific `CommandID` to three critical, asynchronous data streams: Terminal Output, Filesystem/Process Deltas, and OS/Kernel Logs.

By mapping the process tree of a given command, the engine filters out background OS noise and isolates the exact events triggered by the user's action.

```
pub struct EventCorrelator {
    pub command_id: uuid::Uuid,
    pub active_pid: u32, // The root shell PID executing the command
    pub timestamp_range: (DateTime<Utc>, DateTime<Utc>),
}

impl EventCorrelator {
    /// Fetches target OS logs (journalctl / macOS unified logging) matching the
    /// Filters results strictly to the process tree originating from `active_pic
    pub fn fetch_and_filter_logs(&self) -> Result<Vec<OSLogEntry>, CorrelationErr
        // Implementation delegates to platform-specific log readers
    }

    /// Synthesizes the raw logs, filesystem deltas, and stdout into a unified im
    pub fn generate_impact_report(&self) -> SystemImpact {
        // ...
    }
}
```

2. Hardware Abstraction Layer (HAL)

To achieve true system introspection without relying on brittle shell scripts, Terminal Buddy taps directly into OS-level APIs using highly optimized Rust crates. This abstraction layer ensures cross-platform compatibility while maintaining microsecond-level polling precision.

- **IO/Process Monitoring (`sysinfo` / `procfs`):** Captures memory bloat, CPU spikes, and identifies zombie processes left behind by failing commands. This is crucial for explaining performance degradations to the user.
- **System Logs (`libsystemd` bindings / `security-framework`):** Bypasses the need to manually grep through `/var/log`. We hook directly into the systemd journal on Linux and the Unified Logging System on macOS to catch kernel panics, permission denials, and driver failures in real-time.
- **File Watcher (`notify`):** Runs a recursive, asynchronous watcher on the current working directory. This catches ephemeral file changes (like lockfiles or temporary build artifacts) that happen too fast for standard polling intervals.
- **Network Tracking (`net-stat`):** Maps outbound network requests and port bindings directly to the CLI tool running. This allows the Buddy to explicitly state, "This command failed because another process is already using Port 8080."

3. API & IPC Schema: The Dual-Protocol Strategy

The Sidecar must serve two entirely different clients: the Rust GUI (which requires ultra-low latency) and the Canvas Dashboard (which requires a standard web protocol).

1. High-Speed Internal Bus (Bincode over Unix Sockets): Used exclusively between the GUI and the Daemon. Bincode allows for zero-copy deserialization, ensuring the UI maintains its strict 120FPS rendering target without being bogged down by JSON parsing overhead.

2. Web Dashboard API (JSON-RPC): The Sidecar exposes a local HTTP/WebSocket server for the React-based Canvas UI. This allows for rich, stateless data fetching and RAG (Retrieval-Augmented Generation) chat queries.

Endpoint Example: `GetTeachableMoment(command_id)`

```
{
  "command": "docker-compose up",
  "system_impact": {
      "files_modified": ["./docker-compose.yml"],
      "cpu_spike_pct": 45.2
  },
  "root_cause_logs": [
      {
```

```
            "level": "error",
            "source": "kernel",
            "message": "TCP: [::]:8080 bound by process 1290 (nginx)"
        }
    ],
    "buddy_explanation": "Port 8080 is already in use by a background Nginx process
}
```

4. Memvid Storage (`.mv2`): The Write-Ahead Log

The Memvid storage engine acts as a continuous "Flight Recorder" for the developer's machine. It appends atomic "Reality Snapshots" in real-time, functioning structurally similar to a Write-Ahead Log (WAL) database. This ensures crash-safety and zero data loss.

```
pub struct MemvidFrame {
    pub frame_type: u8, // 0: Text Grid, 1: System Snapshot, 2: AI Correlation
    pub timestamp: u64,
    pub data: Vec<u8>,  // Zstd compressed payload to minimize disk footprint

    // The 1536-dimensional array is generated locally to ensure privacy,
    // allowing semantic queries like "When did I configure Nginx last year?"
    pub vector_embedding: [f32; 1536],
}
```

When a user opens the Canvas UI to "scrub" their timeline, the Dashboard rapidly decodes these frames to reconstruct past terminal states visually.

5. Security & Privilege Model: Observation Mode

Deep system introspection inherently requires elevated privileges. Terminal Buddy adheres strictly to the Principle of Least Privilege.

- **The Sudo Handshake:** Upon initial installation or startup, Terminal Buddy will request a one-time `sudo` prompt.
- **Capability Dropping:** Instead of running the entire terminal as root (which is highly dangerous), the Sidecar utilizes Linux Capabilities to grant itself *only* `CAP_DAC_READ_SEARCH` (to read logs across user boundaries) and `CAP_SYS_PTRACE` (to monitor other processes). The actual shell where the user executes commands drops back to standard user permissions immediately.

- **Graceful Degradation:** If the user denies elevated privileges, Terminal Buddy falls back to "Basic Emulator Mode," disabling deep system correlation while retaining standard text rendering and local file tracking.

6. Phase 1 Build Directives (Expanded)

The initial engineering sprint must establish the core data pipelines before any UI rendering is finalized.

1. `src/daemon/log_pipe.rs`: Construct an asynchronous, non-blocking channel that streams OS logs into standard Rust enums. Ensure backpressure mechanisms are in place so log floods do not crash the daemon.
2. `src/daemon/state_diff.rs`: Implement memory-mapped data structures for Pre-Flight and Post-Flight system states. Write the diffing algorithm that efficiently calculates the delta between two massive system snapshots.
3. `src/correlator/mod.rs`: Build the core loop that ties Shell PIDs to the `log_pipe` output based on microsecond-precision timestamps.
4. `src/ipc/socket.rs`: Establish the dual Unix Domain Sockets: one configured for raw